TIPS & TRICKS

How to show mounted filesystems

If your *df* output is not showing all mounted filesystems, then here is what you can do. The *df* command pulls the information from the file */etc/mtab*. Sometimes the *mtab* file is corrupted, missing or has incorrect entries. So the *df* command output shows up incorrectly. However, the file */proc/mounts* keeps track of all the *mountpoints* correctly.

There are two workarounds for this issue.

a) Reboot the server and the file */etc/mtab* will be recreated with the correct entries.

b) If rebooting is not an option, then create symlink */proc/mounts* to */etc/mtab*.

```
ln -sf /proc/mounts /etc/mtab
```

—Krishna Murthy Thimmaiah, krishnamurthy.gt@gmail.com

A Python program for the recovery of deleted files from Trash

Here is a small Python code snippet that can recover files from Trash:

```
import os
import re

path="/home/gurukul/.local/share/Trash/files"
infopath="/home/gurukul/.local/share/Trash/info"
dirlist=os.listdir(path)   #list of file which present in file
folder
directory=[]
popis=""
for fname in dirlist:
    directory.append(fname)
    popis=popis + " " + fname
print popis
fname=raw_input("\nEnter the file name which toyou want to
recover")

a=open(infopath+"/"+fname+".trashinfo","r")

for line in a:
    if "Path=" in line:
        ab=re.findall(r'/.*',line)
destipath=str(ab)
destipath= destipath.lstrip('[')
destipath=destipath.rstrip(']')
destipath=destipath[:-1]
destipath=destipath[1:]
print "destination path is"+ destipath
file1 = open(path+"/"+fname,"r")
file2 = open(destipath,"w")
file2.write(file1.read())
file1.close()
file2.close()
print "files is recovered to desination"
os.remove(path+"/"+fname)
os.remove(infopath+"/"+fname+".trashinfo")
```

—Rajiv Bhandari, raju.r.bhandari@gmail.com

Simulating the *wc -l* command using *sed*

Myriad tasks can be done with the *sed* command. The one liner below will simulate the *wc -l* command which counts the number of lines present in the file.

```
[mickey]$ sed -n '$ =' file.txt

16
```

Let us demystify the above command. But before that, let's understand the working of the *sed* command. *sed* reads a line from the input file into a pattern buffer (the internal buffer used by *sed*), applies commands (if any) on the pattern buffer, and finally prints the modified line on the standard output stream.

Here, the '=' command prints the line number followed by its contents, '$ =' prints the last line number and its contents, and the '-n' option suppresses the default printing of the pattern buffer. Hence it displays only the last line number. Isn't it interesting?

—Narendra Kangralkar, narendrakangralkar@gmail.com